



Getting started with mqTrains Signal Mast Board

mqTrains OVERVIEW

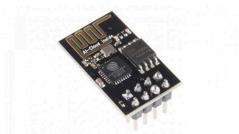
ESP8266 microprocessors are an inexpensive option for controlling model railroad accessories such as turnouts, sensors and signals. They connect quite easily to JMRI using the ESP's WiFi interface and the MQTT messaging protocol. ESP01s can be purchased from online stores for about a couple of bucks each.

mqTrains sketches have been developed or are in development to handle some common model train requirements such as turnout control, signalling and occupancy detection. Visit [mqTrains.com](https://mqtrains.com) for more information.

ESP devices only use the 2.4 GHz WiFi band. Some common domestic WiFi routers are limited to 16 or 32 concurrent sessions. Check your router specifications if you intend to have a high number of these devices.

mqTrains Signal Mast Board

The mqTrainsMast sketch drives a single string of NeoPixels (WS2811 or WS2812) from an ESP01. Groups of consecutive pixels in that string form **signal masts** that display **aspects** (Clear, Stop, Approach, ...) commanded over MQTT. Lights fade in and out, and aspects such as flashing yellow toggle automatically.



- Up to 6 masts per board on items /01 to /06, up to 50 pixels per string
- Colors, aspect names, mast types and wire order are all configurable on the web pages
- WS2811 signal boards (yellow LED on the blue channel) and WS2812 RGB pixels are both supported – select the Pixel Type on the Advanced page

LET'S GET STARTED

These are the main steps to get started with this mqTrains board. More complex features are covered in

the [mqTrains User Reference](#).

1. Install the MQTT Server Software

Linux, Mac and Pi users:

We recommend using a Raspberry Pi for the MQTT Server Software. The Pi Zero W is quite adequate and inexpensive. Follow the vendor instructions to set up the Raspberry Pi. To install Mosquitto Server, use the commands:

```
sudo apt-get update
sudo apt-get install mosquitto mosquitto-clients
```

Mosquitto version 2 (since December 2020) uses tighter security rules. Create this config file:

```
sudo nano /etc/mosquitto/conf.d/mosquitto.conf
```

Insert these two lines and then press ctrl-o <enter> and ctrl-x

```
listener 1883
allow_anonymous true
```

and restart mosquitto with

```
sudo service mosquitto restart
```

Windows users:

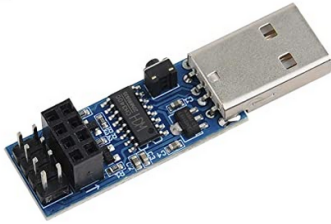
Mosquitto can also be installed on most Windows computers. Refer to the [Mosquitto website](#) for details. If you install on Windows, you will most likely need to grant Firewall permission. Instructions for granting permission with the Windows Defender Firewall are shown at the end of this document. If you have a Firewall with non-Microsoft security software, refer to the vendor instructions.

2. Install mqTrains on your ESP01

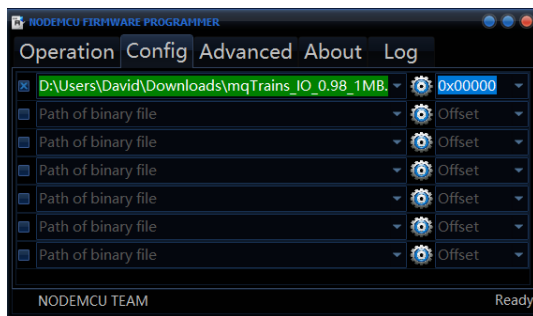
Download the mqTrains binary file from here:
[mqTrains.com/downloads/](https://mqtrains.com/downloads/)

Windows users: Download the [nodemcu-flasher](#) program from the appropriate link: x64 version (most common) or x32 version (for old 32-bit computers).

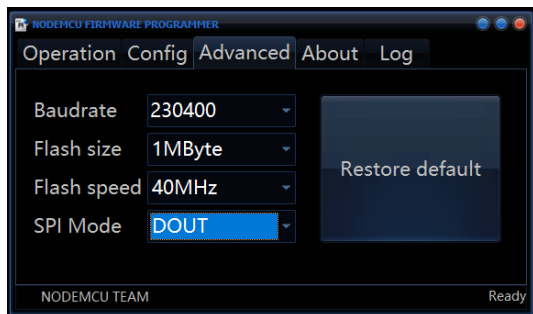
The ESP01 requires an adapter to connect to a computer. We recommend the one shown here. Point the ESP01 towards the USB connector.



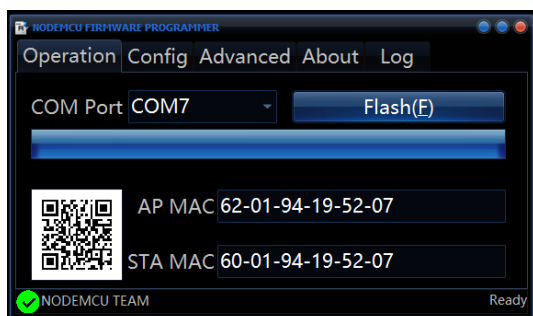
Run the installer (double click the exe file downloaded), select the Config tab, click on the wheel in the first row, then navigate to and select the binary file you downloaded.



Select the Advanced tab and select DOUT for SPI Mode:



On the Operation tab, the correct COM Port will most likely be selected for you, change if need be, then click on Flash to start the upload. The blue bar will show the upload progress which takes about 3 minutes. Wait for the green tick and "Ready" message at the bottom of the screen.



Linux, Mac and Pi users:

```
sudo apt-get update
sudo apt-get install python python-pip
sudo pip install esptool
```

Then, while in the folder with the .bin file, run (all in one line and with the correct serial port, ttyUSB0 shown here):

```
esptool.py --chip esp8266 --port /dev/ttyUSB0
--baud 460800 --before default_reset --after h
ard_reset write_flash 0x0 mqTrains_Mast_x.xx_1
MB.bin
```

3. Wiring your ESP01

Connect a 3.3 Vdc supply to the ESP01 and 5 Vdc to the pixel string. The pixel data-in line connects to the ESP01's GPIO2 pin (the pin is configurable on the Mast Hardware page). Keep the data lead short and share the grounds of both supplies.

Wire the masts one after another along the string: the first mast uses the first pixels, the next mast the following ones, in the order set on the Signal Masts page.

4. Configuring your ESP01

Using a smartphone or computer, look in your available WiFi networks list for the mqTrains Access Point (AP) network service started by the ESP. The default SSID name is "mqTrains-4001" with the Key or password set to "mqtrains".

Once connected to the ESP's AP, use a web browser to connect to the mqTrains configuration page using <http://2.2.2.1>.

You can use these QR codes to connect to the default Access Point and web page. (The 2nd QR code for the web page won't work if your app needs the internet to decode, like Google Lens.)



WiFi mqTrains-4001



Web Page 2.2.2.1

Select the Quick Start option from the main menu and change each of these values to suit your needs.

Set a 4 digit Serial Number for the board. If you have more than one mqTrains board, each should have a unique serial number. This will be used as part of the MQTT messaging. We recommend not leaving the default value, since the next new board will also be 4001. Some use 4000s for signal boards to keep them apart from turnout and sensor boards.

Set the IP address of the computer on which your MQTT Server (Broker) is installed and the port number it uses. 1883 is the default port.

Set the base portion of the MQTT topic. The full topic will have the board serial number and the two digit item number appended to the base. For example, using the default base topic, the full MQTT topic for mast number /01 on board number 4001 will be myLayout/mast/4001/01.

Set the WiFi SSID and Key to connect to your desired WiFi service. Keep in mind that the WiFi service will need to provide you access to the MQTT Server whose IP address you entered earlier.

Press SAVE+REBOOT to save all these values to the ESP's internal storage and restart it using those values.

Using a utility such as MQTT Explorer or an MQTT subscribe line command, look for messages for topics beginning with the Base Topic. Expect to see a message like "192.168.1.10 is here!" This is the IP Address of the ESP on your WiFi network. Use this IP address in your browser to connect to the configuration page to complete the set up or to make changes. The AP will no longer be needed and will shut down once the ESP is able to post a message to the MQTT Server.

Prefer a serial cable? These boards can also be provisioned over the serial port (115200 baud): send `:h;` for the command list, then for example `:Bserial 9001;`, `:Bbase myLayout/relay/;` and `:Bsskr0000 MySSID MyWiFiKey;` followed by a reboot.

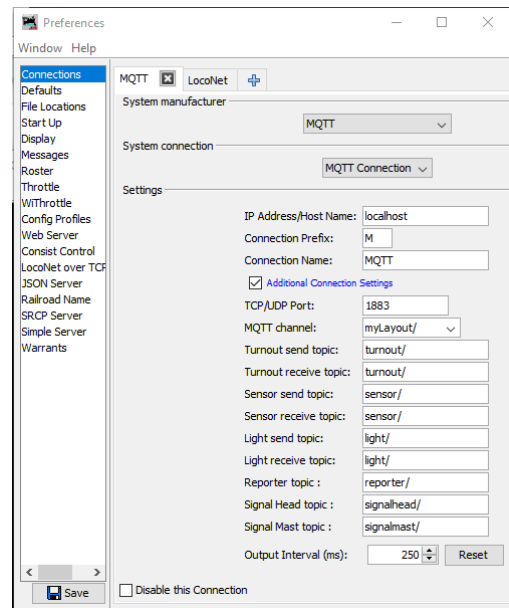
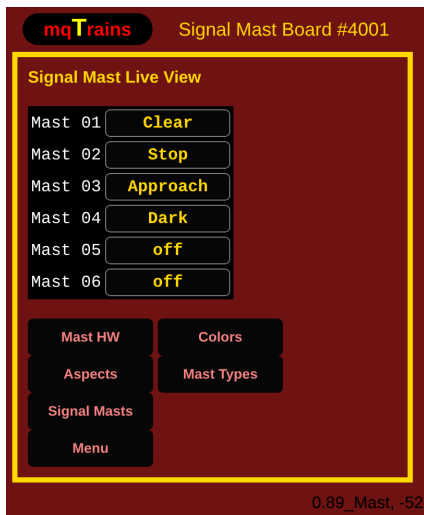
5. Configure the masts

Work down the mast pages on the main menu:

- **Mast Hardware** — pixel pin and flash time for the string.
- **Colors** — the palette, one letter per color. Both yellows are always available: y is a true RGB yellow for WS2812 pixels and Y is the WS2811 signal board's yellow LED wired to the blue channel. Tune the mix to suit your LEDs.
- **Aspects** — the aspect names. The name is the MQTT payload, matched case-insensitively (Clear, Stop, Approach, Dark, ...).
- **Mast Types** — what each kind of mast looks like: how many lights, and per aspect one color sequence per light. A sequence of more than one letter toggles, so yx is flashing yellow (yellow ↔ dark) and rg alternates red/green. The order of the lights also sets the wire order — a red-over-green mast can have the red pixel earlier or later in the string.
- **Signal Masts** — the mast instances: which type each mast is, whether it is enabled, and its position in the string. PUSH applies a setup live for testing; SAVE persists it.

ID	Ena	Mast Type	Order
/01	☒	(1px) 1 3-col light	1
/02	☒	(1px) 1 3-col light	2
/03	☒	(3px) 3 3-col lights	3
/04	☒	(3px) 3 3-col lights	4
/05	☐	(1px) 1 3-col light	5
/06	☐	(1px) 1 3-col light	6

Mast aspects can be monitored using the Live View page. Tapping an aspect advances the mast to its next configured aspect and publishes the matching MQTT message — handy for checking your wiring before JMRI is involved.



JMRI Connections (v 4.21.2)

6. Set JMRI to talk with your ESP01

In JMRI, using Edit -> Preferences -> Connections, create an MQTT connection with the settings as shown, and especially make sure you specify the same host on which you have installed the MQTT Server as you provided to the ESP. You can specify the host name, or the IP address as you did on the ESP01. If the MQTT server is running on the same computer as JMRI, you can also use localhost as shown.

Point the JMRI Signal Mast topic (see the Connections screenshot) at your base topic. Each mast then follows the aspect names sent to 4001/01...4001/06. Any unmatched payload shows the mast's Unknown aspect, so typos are easy to spot on the layout. See the mqTrains-UserReference.pdf for signal system details.

Try mqTrains from the command line

The mqTrains web pages allow you to see and change mast aspects. On a command line, with the mosquitto tools installed, you can see the messages being published while mqTrains is connected to the MQTT Server:

```
prompt> mosquitto_sub -v -h localhost -t "myLayout/#"
or in Windows
"c:\Program Files\mosquitto\mosquitto_sub.exe" -v -h localhost -t "myLayout/#"

-v means verbose - show the full message (topic and payload).
-t means topic
-h means the MQTT Server hostname or IP address
-r means retain the message in the Server.
-m means message, aka payload.
```

You can publish aspect names to the MQTT Server and watch the mast change; retained messages (-r) mean the mast restores its aspect after a power cycle:

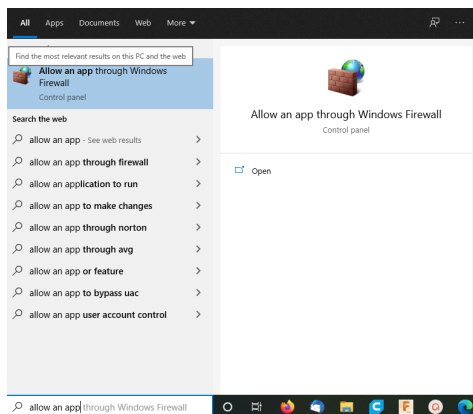
```
prompt> mosquitto_pub -h localhost -r -t "myLayout/mast/4001/01" -m "Stop"
prompt> mosquitto_pub -h localhost -r -t "myLayout/mast/4001/01" -m "Approach"
prompt> mosquitto_pub -h localhost -r -t "myLayout/mast/4001/01" -m "Clear"
```

Note that xxxx/00 is used for board specific things (see the mqTrains-UserReference.pdf for more information) and the masts are from xxxx/01 to xxxx/06.

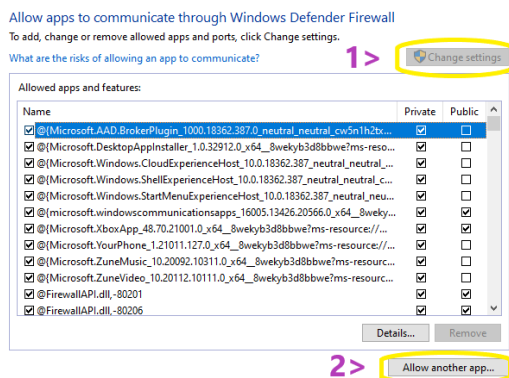
Burning through the Windows Firewall

If you install MQTT Server Software (mosquitto) on a Windows computer, such as your JMRI machine, you may find that the ESP nodes cannot connect to the MQTT Server due to the Windows Defender Firewall blocking WiFi traffic coming into the computer. This section describes how to alter the Firewall to allow MQTT messages to pass through.

Type "allow an app" into the Windows search bar. The "Allow an app through the Windows Firewall" will appear.

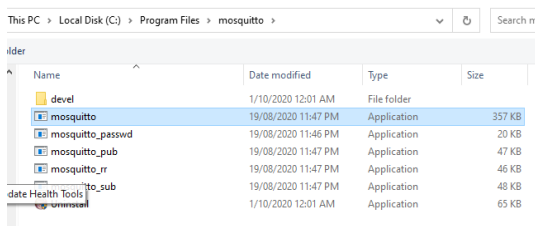


Click on this and then, on "Change Settings" and "Allow another app".

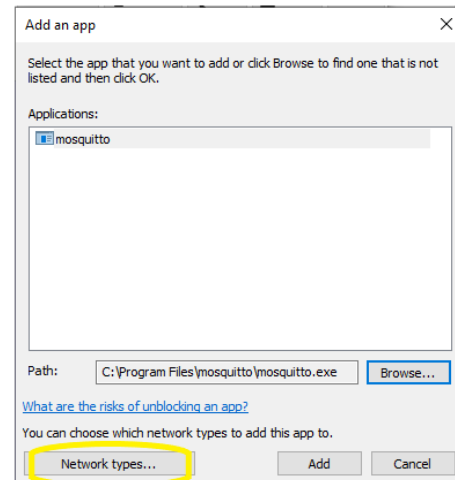


With Browse, select mosquitto.exe from C:\Program Files\mosquitto

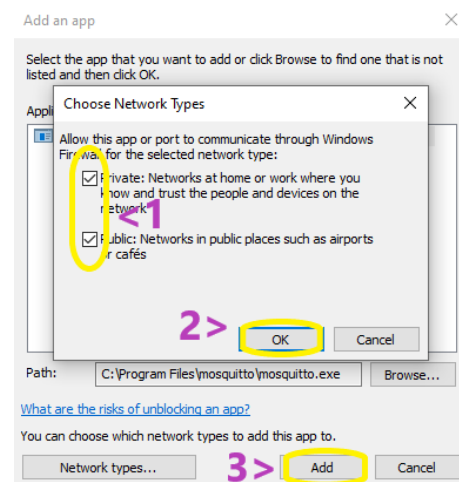
(32-bit machines would have X86 in that folder name)



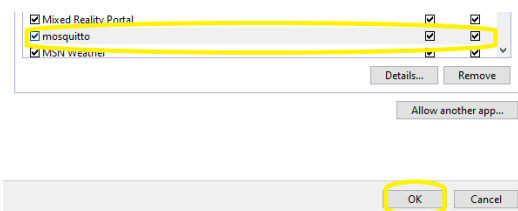
And click on "Open", then click on "Network types".



Depending on how your WiFi network is configured, you may require both Private and Public options, so enable both anyway. Then click on "OK" and "Add".



You will then see mosquitto added to the list. Now press OK to finish.





Credits

mqTrains uses, without modifying any of their respective code as available on github:

ArduinoJSON by Bernoit Blanchon

Arduino Queue by Einar Arnason

PubSubClient by Nick O'Leary

LittleFS file system from [earlephilhowe](#)

ESPAsyncTCP and ESPAsyncWebServer from [me-no-dev](#)

Adafruit NeoPixel